Task	Label	Dev Sample Query: [premise] ⇒ [hypothesis]
I	`Entail`	George Bush was the Governor of Texas ⇒ George Bush is a politician from Texas
$I_{RandPrem}$	`No-Entail`	George Bush resided in Texas ⇒ George Bush is a politician from Texas

Table 2: From the original dataset task (I) we derive the Random Premise task ($I_{RandPrem}$), respecting type-constraints. A random premise is highly unlikely to entail the hypothesis, so samples are relabeled `No-Entail`.

Task	Label	Dev Sample Query: [premise] ⇒ [hypothesis]
I	`Entail`	India exports tons of rice ⇒ India exports rice
I^{GenArg}	`Entail`	location X exports tons of food Y ⇒ location X exports food Y
$I^{RandArg\downarrow}$	`Entail`	Sloterdijk exports tons of oatmeal cookies ⇒ Sloterdijk exports oatmeal cookies
$I^{RandArg\uparrow}$	`Entail`	Helsinki exports tons of Granny Smith ⇒ Helsinki exports Granny Smith

Table 3: An original dev sample (I) is transformed by insertion of entity types (I^{GenArg}); by real entities sampled from the 5% least frequent in NewsCrawl ($I^{RandArg\downarrow}$); and also from the 5% most frequent ($I^{RandArg\uparrow}$).

Transformations involve first identifying the types of entities in statements, in order to constrain entity or predicate replacements. We type each entity with one of the 48 FIGER types (Ling and Weld, 2012), such as "person," "location," etc. First, an entity linker (Nguyen et al., 2014) identifies the Freebase ID (Bollacker et al., 2008) for an entity, from which we then obtain its FIGER type; we assign a default type "thing" in failure cases.

The Random Premise Task ($I_{RandPrem}$) replaces the original premise predicate with a random predicate, while maintaining the same entity arguments. This manipulation produces a dataset in which all samples are labeled `No-Entail`, since two randomly paired predicates are very unlikely to be related by entailment. Thus, positive decisions by the model are false positive hallucinations.[5]

To maintain naturalness and grammaticality, we constrain a new predicate to have argument slots of the same types as the original premise. For example, "[medicine] is indicated for patients with [disease]" is swapped for "[medicine] does not cure [disease]". We source candidates from dev set premises satisfying the target type-constraints, and sample uniform randomly. We map the original entities to their respective slots in the new premise. Examples are shown in Table 2. $I_{RandPrem}$ is a good test of model reliance on propositional memory, since we prevent entailments while maintaining the attestedness of conclusions (hypotheses).

The Generic Argument Task (I^{GenArg}) replaces original entities with unique FIGER-typed identifiers, e.g. "location X" and "food Y." By masking the identities of entities, this test is designed to remove entity information while maintaining the same entailment label, as a baseline control setting. We append unique identifiers (e.g. "X," "Y") to allow tracking of entity slots across the premise and the hypothesis.

The Random Argument Task ($I^{RandArg}$) replaces original entities with other real, random entities of the same FIGER-type. Like I^{GenArg}, this test is designed to create novel strings by modifying statements without changing entailment labels. But now we test model sensitivity to added extraneous information. Examples are shown in Table 3.

We use entity type constraints here to ensure polysemous predicates maintain the same sense. For example, a different sense of *run* is used in "[person] runs [organization]" vs. "[person] runs [software]", but between different entities of the same type, the same senses are used, so the exact entity IDs do not affect entailment labels (Yarowsky, 1993). We source new entities from NewsCrawl (Barrault et al., 2019), a decade-long span of multi-source news text, in which entities are typed as above. We sample new entities uniform randomly from the 5% least common entities in NewsCrawl ($I^{RandArg\downarrow}$), and the 5% most common ($I^{RandArg\uparrow}$). We insert the sampled entities while preserving the rest of each statement.

4 Querying Models with Prompts

4.1 Models

LLaMA is a recent LLM model family which rivals or surpasses GPT-3 performance while being open to scientific study. A range of model sizes

[5]We manually inspected the generated random premise entries for the Levy/Holt dataset to verify this: we found 86.6% of entries are successfully non-entailing, 3.8% undecided cases, and only 9.6% are unintended true entailments.

are provided, and we test the largest **LLaMA-65B** model. LLaMA is not fine-tuned. In preliminary experiments on the Levy/Holt dataset, we found two popular fine-tuned LLaMA variants, Alpaca (Taori et al., 2023) and Vicuna (Chiang et al., 2023), perform similarly to LLaMA base models and underperform LLaMA-65B, so we leave them out of further experiments.

GPT-3 Series models are closed to deep scientific review (Brown et al., 2020), though they are a widely-used comparison for their performance, and have been reasonably well-studied. We evaluate on **text-davinci-003 (GPT-3.5)**, as it is the largest, and has undergone instruction- and RLHF-finetuning, enabling interesting comparisons.

PaLM is larger than GPT-3, which often claims state-of-the-art on evaluation datasets. We use the largest **PaLM-540B** base model, which is also only pretrained, so it serves as a further comparison point to LLaMA.

Later GPT models (like text-davinci-003 in our experiments) have been pre-trained and fine-tuned, while base LLaMA and PaLM have only undergone pre-training, so their contrast indicates what stage of training is responsible for the phenomena we study. Our aim is not to judge which LLM is superior, but to show the common sources of hallucination they share.

We also omit models superseded in performance by LLaMA (e.g. OPT, GPT-J, etc.), as well as products that are closed to scientific review (e.g. GPT-4, Bard, etc.)[6].

4.2 Prompt Design and Evaluation

Formatting of test samples is done by inserting the premise and hypothesis into a prompt template, which is used to query the model in natural language. Following this, we append a three-way answer choice: A) Entailment, B) Neutral, C) Contradiction, following the typical format in NLI (Bowman et al., 2015).

Selection of the prompt template used in test is decided by the highest AUC obtained on the respective dev set. We try 8 promising templates including 5 from Schmitt and Schütze (2021), also used in other NLI work[7] (Webson and Pavlick, 2022).

[6]We include an analysis of GPT-4 in Appendix F.

[7]See Appendix A for details on prompt template selection.

Ideally, an LLM with advanced language understanding ability could perform inference in zero-shot without annotated examples, which would raise confidence that this faculty is ready for downstream tasks. To this end, we examine each LLM in zero-shot (detailed in Appendix A), but they exhibit severely degraded, even near-random performance.

We turn to few-shot, and hand-annotate a minimal 4 examples in the style of the template, with added explanations about why the given answer is correct for each example. These examples are prepended before the query (see Appendix A for an example). Our goal is to study model behavior as conditions change, not to maximize the score on any particular dataset. Therefore, we use a minimal 4-example setup, which we find is capable of evoking positive responses from all three LLMs on each dev set, across most templates.

Scoring is done by converting choice A into `Entail` and collapsing both B and C choices into `No-Entail` to align with Levy/Holt and RTE-1 annotation. For behavioral experiments in §5, §6, and §7, we score the model solely based on its textual response. All models successfully choose one of A/B/C on all dev questions, showing compatibility with the QA format.

For the analysis in §8 which measures model performance across confidence thresholds, we convert the letter choice to a probability with the mapping:

$$S_{\text{ent}} = 0.5 + 0.5 * \mathbb{I}[\text{tok} = \mathbf{A}] * S_{\text{tok}} - 0.5 * \mathbb{I}[\text{tok} \in \{\mathbf{B}, \mathbf{C}\}] * S_{\text{tok}}$$

Where $\mathbb{I}$ is the indicator function, and S_{ent} estimates the probability of `Entail` from a textual output ($0 \leq S_{\text{ent}} \leq 1$) with token probability S_{tok}. The linear transformation preserves the ordering of model confidences, which is sufficient for calculating a precision-recall curve.

5 Experiment 1: Attestation Bias

We begin our experiments by assessing LLMs' reliance on their propositional memory of training text by conditioning each model's entailment task predictions I on its own predictions of attestation Λ. We do this by comparing estimated probabilities of predicting `Entail` conditioned on whether the hypothesis is predicted `Attested` or not.

Further, we test a setting which controls for the possibility that original Levy/Holt entailments may coincidentally refer to attested facts, which could

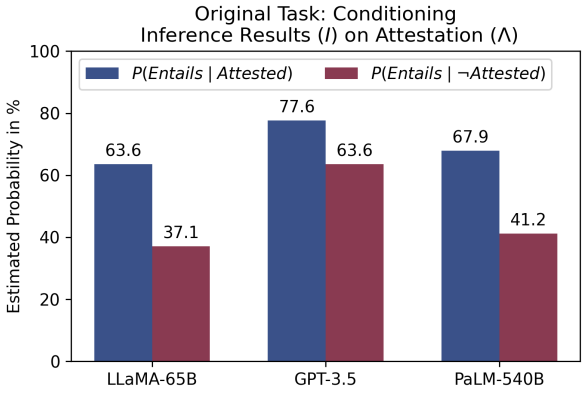

Figure 1: Exp-1. Estimated probability of predicting `Entail` for **original** entries in Levy/Holt, conditioned on LLMs' attestation of hypotheses (Λ). This setting is intuitive but may be subject to spurious correlations.

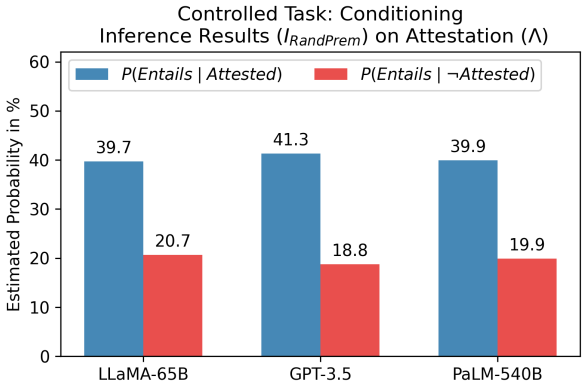

Figure 2: Exp-1. Estimated probability of predicting `Entail` for **Random-Premise** entries in Levy/Holt, conditioned on LLMs' attestation of hypotheses (Λ). Now, predicting `Entail` is false positive hallucination (lower is better). Models are sensitive to attestation, and hallucinate more when the hypothesis is attested.

lead to spurious correlation between inference and attestation scores without clearly demonstrating use of memory versus true entailment. This controlled setting is the random premise task $I_{RandPrem}$, which converts entailments into non-entailments without altering the hypothesis. An ideal model capable of drawing inferences from information in context should detect that in the $I_{RandPrem}$ task it is no longer possible to infer the hypothesis based on the premise (even if the hypothesis is itself attested in training), and never predict `Entail`. Thus, in $I_{RandPrem}$, all `Entail` predictions are assumed to be false positive hallucinations.

5.1 Results

With I, $I_{RandPrem}$ and Λ predictions acquired as described in §3.1, we present the conditional probabilities in Figures 1 and 2. It is clear that a model's memory about the hypothesis plays a part in its predictions of the hypothesis given a premise, either related or random.

For I, we observe significantly higher probability of predicting `Entail` when the hypothesis is `Attested`. In the random premise task $I_{RandPrem}$, this trend continues. LLaMA, GPT-3.5, and PaLM, respectively, show a 1.9x, 2.2x, and 2.0x higher chance of falsely predicting that a random premise `Entail`s the hypothesis if it already predicts the hypothesis is `Attested`. This false positive hallucination and its impact on NLI performance is investigated further in §8.

This behavior is observed across model families (LLaMA, GPT, and PaLM), establishing that it is due to pretraining rather than Instruction-tuning or RLHF, since LLaMA and PaLM have only undergone pretraining. This behavior is undesirable, because model predictions on NLI tasks should be based solely on general language understanding, not prior knowledge. We may conclude that memory of training data is a significant contributor in LLM inference, and may be an important source of hallucination.

5.2 Implications for Real Applications

Using prior knowledge as part of language inference has bad implications for the use of LLMs in real applications. We offer an example scenario of a question-answering task where user questions are answered from a Knowledge Base (KB). In typical formulations of this task, if a statement in the KB (premise) entails a user query (hypothesis), the premise may be formulated into an answer. Consider a KB such as a legal document or HR rulebook. Assume that the text is prepended to the user query and presented to the LLM, as in other works (Srinivasan et al., 2022). Given our findings, we might observe the LLM hallucinating answers to questions using information which is not presented in the KB, but may have been read by the LLM in text from other sources during pretraining. These answers could be illogical, contradictory, and could misrepresent the views of the KB, or other harms. Such poor use of in-context learning has already been observed in specific domains like medicine (Jimenez Gutierrez et al., 2022).

In general, this is a risk for LLMs which (a) are deployed for tasks like QA by feeding novel text (e.g. a legal document) in-context as part of the user query, and (b) are trained on datasets which are